

Lekcja 13 - Spark Internals i koszt wykonania

2026-09-10

Data Engineering Course

Spark Internals i koszt wykonania

Nie wystarczy, że job daje dobry wynik.
Musisz jeszcze rozumiec, dlaczego trwa tyle, ile trwa.

■ Co ma być lepsze niż month_02

Month 02 uczył pisanie pipeline logic. Ta lekcja uczy:

- kosztu fizycznego planu,
- myślenia o rozkładzie pracy,
- diagnozy bottlenecków,
- decyzji wykonaniowych przy skali.

■ Problem biznesowy

Pipeline działa dla 500k rekordów. Przy 50 mln robi się problem.

5 min -> 20 min -> 2h

To nie musi znaczyć błędu w SQL. To często znaczy:

- za dużo shuffle,
- skew,
- zły join,
- zła liczba partycji,
- bezsensowny cache.

■ First principle

Wolny Spark job to zwykle problem fizycznego planu wykonania,
a nie logicznej poprawnosci query.

To jest roznica miedzy:

- "umie napisac PySpark",
- a
- "umie utrzymac pipeline na duzych danych".

```
Job  
  -> Stages  
    -> Tasks  
      -> Partition data
```

Zapamiętaj:

Stage kończy się tam, gdzie Spark musi przetasować dane.

■ Narrow vs wide

```
Narrow:  
  filter  
  select  
  withColumn
```

```
Wide:  
  groupBy  
  join  
  distinct  
  orderBy
```

Wide transformations zwykle uruchamiają shuffle.

■ Co to jest shuffle

```
serialize  
-> write  
-> network  
-> read  
-> deserialize
```

To jest drogie, bo zużywa jednocześnie:

- CPU,
- dysk,
- sieć,
- synchronizację stage.

■ Skąd wiesz, że problemem jest shuffle

Szukasz:

- `Exchange` w `explain()`,
- dużego shuffle read / write w UI,
- stage, który trwa dużo dłużej niż reszta.

Nie zgadujesz. Patrzysz na dowód.

■ Co to jest skew

Jeden klucz ma dużo więcej danych niż reszta.

Przykład:

```
US = 180000  
PL = 10000  
DE = 10000
```

Skutek:

jeden task pracuje bardzo długo,
reszta workerów już czeka

■ Jak wykryć skew

```
df.groupBy("country").count().orderBy(F.desc("count")).show()
```

I potem w UI:

Task duration: czy są duże outliery?

Spark performance zaczyna się od rozkładu danych, nie od configów.

■ Broadcast join

```
df_large.join(F.broadcast(df_small), "customer_id")
```

Kiedy ma sens:

- jedna tabela jest mała,
- lookup jest prosty,
- koszt broadcastu < koszt shuffle obu tabel.

■ Kiedy broadcast nie pomoze

- lookup nie jest juz maly,
- memory executorow jest napiete,
- po joinie i tak robisz ciezka agregacje,
- problemem jest skew, nie sama strategia joina.

To nie jest "zawsze lepszy join".

Adaptive Query Execution robi runtime tuning:

- coalesce po shuffle,
- lepszy join strategy,
- split skewed partitions.

Praktyczna zasada:

Najpierw włącz AQE.
Dopiero potem wprowadzaj ręczne fixy.

■ Partitions

Za mało partycji -> słaba równoległość
Za dużo partycji -> scheduler overhead i małe taski

Musisz umieć odpowiedzieć:

Czy obecna liczba partycji odpowiada wolumenowi danych?

■ Repartition vs coalesce

```
repartition(N)
- robi shuffle
- lepszy rowny rozklad

coalesce(N)
- zwykle bez shuffle
- tansze przy zmniejszaniu
```

Dobra decyzja zalezy od celu, nie od przyzwyczajenia.

■ Cache i persistence

Cache ma sens tylko wtedy, gdy:

```
ten sam DataFrame liczysz kilka razy,  
a jego przeliczenie jest drogie.
```

Cache szkodzi, gdy:

- dane sa uzyte raz,
- memory jest ciasne,
- powoduje spills i GC pressure.

■ Recipe diagnozy

1. `explain(mode="formatted")`
2. Exchange?
3. Join strategy?
4. Skew?
5. AQE?
6. Partycje?
7. Cache?
8. Write layout?

To jest lepsze niz losowe zmienianie configow.

■ Mini case

Masz flow:

```
orders  
  -> join customers  
  -> groupBy country  
  -> write parquet
```

Pytania:

1. Gdzie jest shuffle?
2. Czy customers to broadcast candidate?
3. Czy country ma skew?
4. Czy write nie tworzy za duzo plikow?

■ Co oddajesz po lekcji

```
homework/lesson_13/  
  01_explain_walkthrough.md  
  02_join_strategy_benchmark.py  
  03_skew_investigation.md  
  04_skew_fix.py  
  05_runtime_tuning_notes.md  
  06_interview_answers.md
```

■ Tutoriale po lekcji

1. Apache Spark Tuning Guide
<https://spark.apache.org/docs/latest/tuning.html>
2. Databricks tutorial: Build an ETL pipeline with Apache Spark on the Databricks platform
<https://docs.databricks.com/aws/en/getting-started/etl-quick-start>

■ Closing check

Powiedz na głos:

```
Wolny Spark job to zwykle shuffle, skew,  
zły join strategy, zła liczba partycji albo zły  
reuse danych.  
Najpierw diagnoza, potem tuning.
```